

Additionally, if different classes implement the same functionality, it's a good idea to use the same method names. For example, many of the classes in `pygwidgets` have a method named `setValue()` and another named `getValue()`. I'll talk more about why this type of consistency is so important in the next two chapters.

Summary

This chapter provided an introduction to the object-oriented `pygwidgets` package of graphical user interface widgets. We began by discussing default values for parameters in methods, and I explained that a keyword parameter allows for a default value to be used if no matching argument value is specified in a call.

I then introduced you to the `pygwidgets` module, which contains a number of prebuilt GUI widget classes, and showed you how to use several of these. Finally, I showed a sample program that provides examples of most of these widgets.

There are two key advantages to writing classes like those in `pygwidgets`. First, classes can hide complexity in methods. Once you have your class working correctly, you never have to worry about the internal details again. Second, you can reuse the code by creating as many instances of a class as you need. Your classes can provide basic functionality by including keyword parameters with well-chosen default values. However, the default values can easily be overwritten to allow for customization.

You can publish the interfaces of your classes for other programmers (and yourself) to take advantage of in different projects. Good documentation and consistency go a long way toward making these types of classes highly usable.